AIMI LAB2 — EEG Classification

412551030 Yih-Ning Huang

I. Introduction

The objective of Lab2 is to implement EEGNet and apply it to binary motor imagery classification. Specifically, the model is trained to distinguish imagined left-hand movements from imagined right-hand movements using the BCI Competition III Dataset IIIb, which contains two bipolar EEG channels [1]. Electroencephalography (EEG) is a non-invasive technique that records electrical activity associated with the brain through electrodes placed on the scalp. In an EEG-based brain–computer interface (BCI), recorded signals are analyzed to identify patterns related to a user's intent. Motor imagery provides one such source of information: imagining different movements produces patterns that can be used for classification [2]. This task therefore offers an opportunity to study how neural networks extract useful information from EEG recordings. Figure 1 illustrates the basket paradigm and an example two-channel EEG trial. Each trial contains 750 samples per channel, corresponding to approximately 6 s at 125 Hz. The paradigm includes visual cues and feedback, which may contribute to the recorded EEG activity. However, the EEG segment's alignment with cue onset has not been established.

In this lab, five models were evaluated: EEGNet [2], DeepConvNet and ShallowConvNet [3], SCCNet [4], and MBEEGSE [5]. Hyperparameter tuning was conducted by varying batch size and learning rate for each model. Additional ablation studies examined the effects of dropout rate and activation function on classification performance. These experiments aimed to investigate how model architecture and training settings influence learning behavior and testing accuracy.

A. EEGNet and Convolution Design

EEGNet [2] is a compact CNN designed for EEG-based brain–computer interfaces. Our implementation follows the EEGNet-8,2 architecture, with $F_1 = 8$ temporal filters, a depth multiplier of $D = 2$, and $F_2 = 16$ output feature maps. It first applies temporal convolution with a kernel size of $(1, 64)$, followed by depthwise spatial convolution with a kernel size of $(C, 1)$, where $C = 2$ is the number of EEG channels. The spatial layer learns two spatial filters for each temporal feature map, producing sixteen feature maps. A separable convolution then applies depthwise temporal filtering with a $(1, 16)$ kernel and pointwise feature mixing with a $(1, 1)$ kernel. Batch normalization, ELU activation, average pooling, and dropout are used within the convolutional blocks. The two pooling operations use kernel sizes and strides of $(1, 4)$ and $(1, 8)$, respectively. The baseline dropout rate is 0.5, and max-norm constraints of 1.0 and 0.25 are applied to the spatial filters and classifier weights, respectively. Finally, the extracted features are flattened and passed to a two-class fully connected layer.

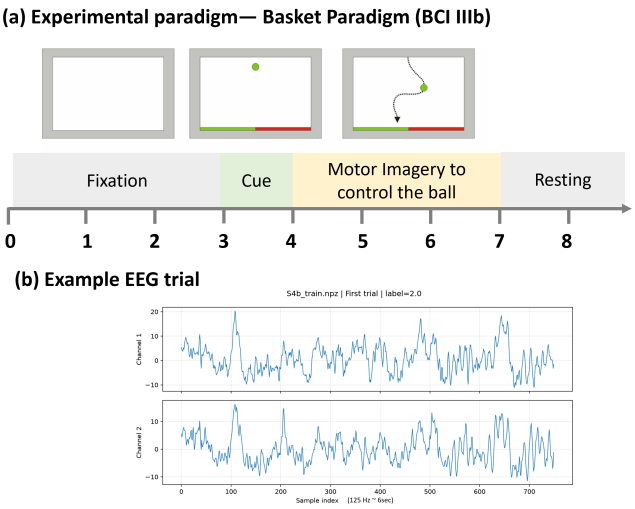


Fig. 1. Illustration of the basket paradigm for BCI Competition III Dataset IIIb [1] (top) and an example two-channel EEG trial (bottom). The EEG segment contains 750 samples at 125 Hz, corresponding to approximately 6 s. The lower plots use their own sample-index axis; their alignment with the upper schematic timeline has not been established.

II. Experiment Setups and Result

All experiments were conducted on an NVIDIA GeForce RTX 5070 Ti GPU. Five models were evaluated on the BCI Competition III Dataset IIIb: EEGNet, DeepConvNet, ShallowConvNet, SCCNet, and MBEEGSE. These models were selected to compare different approaches to spatial and temporal feature extraction, including compact convolution, deeper feature hierarchies, power-based representations, and multi-branch attention.

EEGNet [2] serves as the primary baseline (as described in section Introduction). DeepConvNet [3] uses an initial temporal–spatial convolutional block followed by three additional convolutional blocks, allowing us to examine the effect of a deeper architecture. ShallowConvNet [3] uses temporal and spatial convolutions followed by squaring, average pooling, and a logarithmic transformation to learn representations related to signal power. SCCNet [4] performs spatial filtering before spatio-temporal convolution and power extraction, providing a comparison with models that begin with temporal filtering. Finally, MBEEGSE [5] combines three EEGNet-based branches with different temporal kernel sizes and squeeze-and-excitation blocks to investigate multi-scale feature extraction and channel-wise feature recalibration.

TABLE I
BEST GRID-SEARCH CONFIGURATIONS WITH NATIVE DROPOUT AND ACTIVATION SETTINGS. THE HIGHEST ACCURACY IS SHOWN IN BOLD.

Model	Batch	LR	Epoch	Acc. (%)
EEGNet	128	10^{-2}	2464	88.98
DeepConvNet	64	10^{-2}	1724	89.72
ShallowConvNet	256	10^{-3}	2827	87.31
SCCNet	128	10^{-4}	460	85.00
MBEEGSE	64	10^{-3}	438	**90.65**

Native activations are ELU for EEGNet, DeepConvNet, and MBEEGSE, and square for ShallowConvNet and SCCNet. Native dropout is 0.5, except for MBEEGSE, whose branches use 0, 0.1, and 0.2. Epoch denotes the epoch with the highest testing accuracy within a 3000-epoch run.

TABLE II
PEAK TESTING ACCURACY (%) FOR DIFFERENT DROPOUT RATES. THE BEST RESULTS IN EACH COLUMN ARE BOLD.

Dropout	EEGNet	Deep	Shallow	SCCNet	MBEEGSE
0.1	86.67	85.56	86.39	84.17	90.00
0.3	87.78	87.31	87.22	84.44	90.37
0.5	**88.98**	**89.54**	**87.31**	85.00	**91.76**
0.7	86.67	86.94	**87.31**	**85.28**	91.02
0.9	79.26	76.67	86.48	82.59	84.91

Deep and Shallow denote DeepConvNet and ShallowConvNet. Batch size and learning rate follow Table I. Native activation functions are retained. For MBEEGSE, each tested dropout rate is applied uniformly across all three branches.

TABLE III
PEAK TESTING ACCURACY (%) FOR DIFFERENT ACTIVATION FUNCTIONS. THE BEST RESULTS IN EACH COLUMN ARE BOLD.

Activation	EEGNet	Deep	Shallow	SCCNet	MBEEGSE
ELU	**88.89**	**89.54**	72.96	70.28	**91.76**
GELU	88.24	88.80	73.70	74.44	91.48
LeakyReLU	86.20	88.43	76.85	75.56	90.65
ReLU	87.41	87.78	78.70	78.89	90.83
Square	—	—	**87.31**	**85.28**	—

Deep and Shallow denote DeepConvNet and ShallowConvNet. Batch size and learning rate follow Table I. Dropout is fixed at 0.5, except for SCCNet, which uses 0.7. Square results are matched native-activation controls from the dropout study; dashes indicate untested settings. For MBEEGSE, SE gate activations remain unchanged.

A. Training Configuration

All models were trained using the Adam optimizer with $\beta_1 = 0.9$, $\beta_2 = 0.999$, and $\epsilon = 10^{-8}$. The optimizer weight decay was set to zero. Cross-entropy loss was applied directly to the two-class output logits, without an explicit softmax operation during training.

Model-specific regularization was retained. For EEGNet, max-norm constraints of 1.0 and 0.25 were applied to the spatial filters and classifier weights, respectively. DeepConvNet and ShallowConvNet used max-norm limits of 2.0 for convolutional filters and 0.5 for classifier weights. SCCNet included an additional L2 penalty with a coefficient of 10^{-4} on the weights of its two convolutional layers. MBEEGSE used the spatial constraint convention of its reference implementation and a classifier max-norm limit of 0.25. These constraints were applied after each optimizer update. Because model performance can depend on the training configuration, a separate grid search was performed for each model rather than assuming that one setting would be equally effective for all architectures. Batch sizes of $\{32, 64, 128, 256\}$ and learning rates of $\{10^{-2}, 10^{-3}, 10^{-4}, 10^{-5}\}$ were evaluated, with each configuration trained for 3000 epochs. The learning rate remained constant throughout each run, and no early stopping was used.

The grid search retained each model's native dropout and activation settings. Additional ablation studies compared dropout rates of $\{0.1, 0.3, 0.5, 0.7, 0.9\}$ using the selected batch size and learning rate for each model. Subsequently, ELU, GELU, LeakyReLU, and ReLU were evaluated while fixing each model's dropout rate to the best setting from the dropout study. The native square activation was also retained as a reference for SCCNet and ShallowConvNet. For MBEEGSE, dropout rates were applied uniformly across branches, while activation changes were limited to the EEGNet blocks.

B. Model Comparisons

Table I summarizes the best grid-search results for each model using its native dropout and activation settings. MBEEGSE achieved the highest testing accuracy of 90.65%, followed by DeepConvNet at 89.72% and EEGNet at 88.98%. EEGNet achieved its best result with a batch size of 128 and a learning rate of 10^{-2}. ShallowConvNet and SCCNet achieved 87.31% and 85.00%, respectively. These results show that MBEEGSE performed best under the evaluated configurations, while the compact EEGNet also achieved competitive accuracy.

C. Ablation Study

1) Dropout Rate: Table II compares five dropout rates while keeping each model's selected batch size, learning rate, and native activation unchanged. A dropout rate of 0.5 achieved the highest testing accuracy for EEGNet, DeepConvNet, and MBEEGSE. ShallowConvNet achieved the same peak accuracy with dropout rates of 0.5 and 0.7, whereas SCCNet performed best with 0.7. MBEEGSE achieved the highest overall accuracy of 91.76% with a dropout rate of 0.5. Increasing dropout to 0.9 reduced accuracy for all five models relative to their best settings.

2) Activation Functions: Table III compares ELU, GELU, LeakyReLU, and ReLU using each model's selected batch size, learning rate, and dropout rate. ELU achieved the highest testing accuracy among the tested activation functions for EEGNet, DeepConvNet, and MBEEGSE, reaching 88.89%, 89.54%, and 91.76%, respectively. For ShallowConvNet and SCCNet, the native square activation outperformed all four alternatives. These comparisons replace the square operation while retaining pooling and the logarithmic transformation,

```
EEGNet bs128_lr0.01_donative_actnative_seed42 [100/3000] train=79.44% test=79.63% best=80.19%
EEGNet bs128_lr0.01_donative_actnative_seed42 [200/3000] train=83.06% test=80.28% best=81.39%
EEGNet bs128_lr0.01_donative_actnative_seed42 [300/3000] train=84.72% test=81.20% best=82.78%
EEGNet bs128_lr0.01_donative_actnative_seed42 [400/3000] train=86.02% test=83.80% best=83.80%
EEGNet bs128_lr0.01_donative_actnative_seed42 [500/3000] train=84.91% test=83.80% best=84.72%
EEGNet bs128_lr0.01_donative_actnative_seed42 [600/3000] train=87.22% test=84.72% best=85.56%
EEGNet bs128_lr0.01_donative_actnative_seed42 [700/3000] train=86.48% test=84.54% best=86.57%
EEGNet bs128_lr0.01_donative_actnative_seed42 [800/3000] train=88.06% test=85.74% best=87.13%
EEGNet bs128_lr0.01_donative_actnative_seed42 [900/3000] train=87.96% test=87.13% best=87.59%
EEGNet bs128_lr0.01_donative_actnative_seed42 [1000/3000] train=88.89% test=86.67% best=87.59%
EEGNet bs128_lr0.01_donative_actnative_seed42 [1100/3000] train=88.24% test=85.00% best=87.96%
EEGNet bs128_lr0.01_donative_actnative_seed42 [1200/3000] train=88.15% test=87.04% best=88.24%
EEGNet bs128_lr0.01_donative_actnative_seed42 [1300/3000] train=88.52% test=87.04% best=88.33%
EEGNet bs128_lr0.01_donative_actnative_seed42 [1400/3000] train=90.74% test=87.59% best=88.33%
EEGNet bs128_lr0.01_donative_actnative_seed42 [1500/3000] train=88.15% test=85.83% best=88.33%
EEGNet bs128_lr0.01_donative_actnative_seed42 [1600/3000] train=89.54% test=87.04% best=88.61%
EEGNet bs128_lr0.01_donative_actnative_seed42 [1700/3000] train=90.93% test=86.67% best=88.61%
EEGNet bs128_lr0.01_donative_actnative_seed42 [1800/3000] train=88.70% test=87.31% best=88.61%
EEGNet bs128_lr0.01_donative_actnative_seed42 [1900/3000] train=89.54% test=86.85% best=88.61%
EEGNet bs128_lr0.01_donative_actnative_seed42 [2000/3000] train=88.70% test=86.48% best=88.61%
EEGNet bs128_lr0.01_donative_actnative_seed42 [2100/3000] train=91.76% test=87.41% best=88.61%
EEGNet bs128_lr0.01_donative_actnative_seed42 [2200/3000] train=89.44% test=87.50% best=88.61%
EEGNet bs128_lr0.01_donative_actnative_seed42 [2300/3000] train=90.56% test=87.96% best=88.89%
EEGNet bs128_lr0.01_donative_actnative_seed42 [2400/3000] train=89.63% test=86.39% best=88.89%
EEGNet bs128_lr0.01_donative_actnative_seed42 [2500/3000] train=88.89% test=85.56% best=88.98%
EEGNet bs128_lr0.01_donative_actnative_seed42 [2600/3000] train=90.65% test=87.41% best=88.98%
EEGNet bs128_lr0.01_donative_actnative_seed42 [2700/3000] train=89.91% test=86.85% best=88.98%
EEGNet bs128_lr0.01_donative_actnative_seed42 [2800/3000] train=89.26% test=86.94% best=88.98%
EEGNet bs128_lr0.01_donative_actnative_seed42 [2900/3000] train=91.48% test=88.24% best=88.98%
EEGNet bs128_lr0.01_donative_actnative_seed42 [3000/3000] train=91.30% test=87.13% best=88.98%
[DONE] EEGNet: 88.9815% @ epoch 2464
```

Fig. 2. Screenshot of the highest testing accuracy achieved by EEGNet: 88.98%.

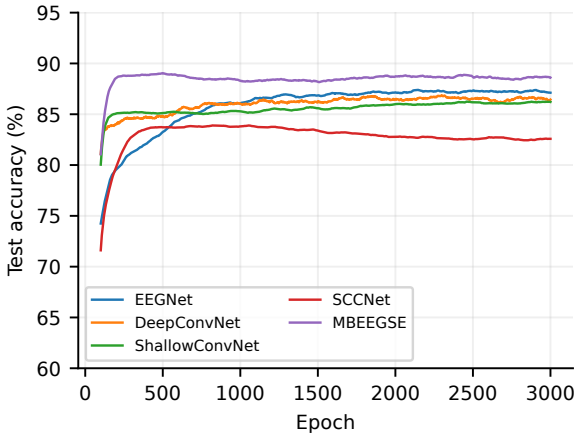


Fig. 3. Testing accuracy trajectories of the five models using their best native-grid configurations. Curves show trailing 100-epoch averages.

and therefore represent changes to their power-based feature extraction pipelines.

D. Highest Testing Accuracy screenshot

Figure 2 shows a screenshot of the highest testing accuracy achieved by EEGNet. EEGNet reached 88.98% with a batch size of 128, a learning rate of 10^{-2}, a dropout rate of 0.5, and ELU activation.

E. Learning Curves

Figure 3 compares the testing accuracy trajectories of the five models using their best native-grid configurations. For readability, the curves are displayed as trailing 100-epoch averages. Reported peak accuracies and best epochs were calculated from the original unsmoothed histories. MBEEGSE reached a relatively high testing accuracy early in training, whereas EEGNet improved more gradually. Their highest testing accuracies were observed at epochs 438 and 2464, respectively. Each model used the batch size and learning rate reported in Table I; therefore, the curves reflect both model architecture and selected training settings.

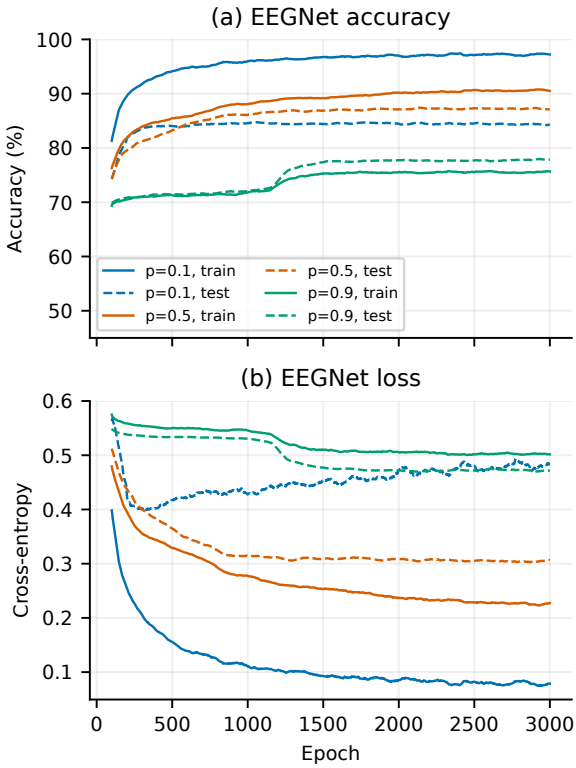


Fig. 4. EEGNet training and testing (a) accuracy and (b) cross-entropy loss for dropout rates of 0.1, 0.5, and 0.9. Curves show trailing 100-epoch averages. Solid and dashed lines denote training and testing, respectively.

Figure 4 presents the training and testing accuracy and loss curves for EEGNet with dropout rates of 0.1, 0.5, and 0.9. A 100-epoch moving average was applied to reduce short-term fluctuations and highlight the overall training trends. Batch size 128, learning rate 10^{-2}, and ELU activation were fixed across these runs. At dropout 0.1, training accuracy continued to increase while testing accuracy plateaued, and testing loss increased after its initial decrease. Dropout 0.5 produced a smaller training–testing gap, whereas dropout 0.9 resulted in lower training and testing accuracy. The implications of these patterns are examined in the Discussion.

III. Discussion

A. Experimental Findings

1) Model Architecture and Classification Performance: MBEEGSE achieved the highest native-grid testing accuracy of 90.65%, increasing to 91.76% after dropout tuning. Its different temporal kernel lengths support multi-scale feature extraction, while squeeze-and-excitation blocks recalibrate feature maps before branch fusion [5]. These mechanisms may explain its strong performance, although their individual contributions were not isolated. Model size alone did not determine accuracy. MBEEGSE contains 6,472 parameters, compared with 266,377 for DeepConvNet and 1,874 for EEGNet. Despite its smaller size, MBEEGSE exceeded DeepConvNet's native-grid accuracy of 89.72%, while the compact EEGNet

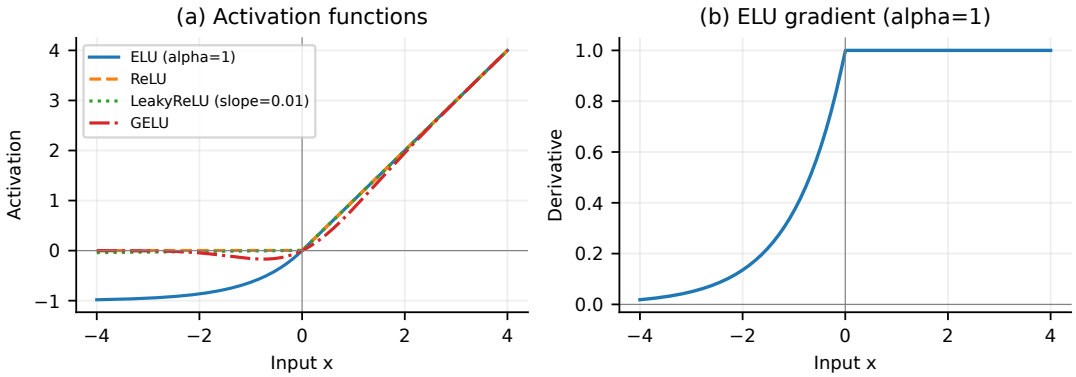


Fig. 5. Analytical activation functions and the ELU derivative. ELU uses $\alpha = 1$ and LeakyReLU a negative slope of 0.01, matching the tested settings. These are mathematical functions, not measured network activations.

achieved 88.98%. These results suggest that suitable feature extraction can be more important than simply increasing model size.

2) Learning Behavior and Hyperparameter Sensitivity: Figure 3 shows that MBEEGSE reached high testing performance earlier, whereas EEGNet improved more gradually. Their peak accuracies occurred at epochs 438 and 2464, respectively, although peak epoch does not directly measure convergence time. The selected settings also varied across models. EEGNet used batch size 128 and learning rate 10^{-2}, while MBEEGSE used batch size 64 and learning rate 10^{-3}. SCCNet selected a learning rate of 10^{-4}, and ShallowConvNet used batch size 256 with learning rate 10^{-3}. Differences in feature scales, nonlinearities, and optimization dynamics may contribute to this sensitivity, supporting model-specific tuning.

3) Dropout, Overfitting, and Underfitting: EEGNet, DeepConvNet, and MBEEGSE performed best at $p = 0.5$, whereas SCCNet preferred $p = 0.7$. ShallowConvNet tied at $p = 0.5$ and $p = 0.7$. For MBEEGSE, a common dropout rate of 0.5 improved the observed peak compared with its native branch rates of 0, 0.1, and 0.2, suggesting that stronger regularization may benefit this multi-branch model.

Figure 4 illustrates the trade-off using EEGNet. At $p = 0.1$, average training and testing accuracy over the final 100 epochs were 97.23% and 84.33%, a gap of 12.90 percentage points. Decreasing training loss accompanied by increasing testing loss further indicates overfitting. At $p = 0.5$, these averages were 90.50% and 87.10%, reducing the gap to 3.40 percentage points. Mean testing loss also decreased from 0.482 to 0.306, indicating a better balance between training fit and testing performance. In contrast, $p = 0.9$ produced 75.67% training and 77.84% testing accuracy, consistent with excessive regularization and underfitting.

4) Activation Functions and Model-Specific Effects: ELU achieved the highest activation-study accuracy for EEGNet, DeepConvNet, and MBEEGSE: 88.89%, 89.54%, and 91.76%, respectively. For MBEEGSE, its advantage over GELU was only 0.09 percentage points, equivalent to one testing trial. Repeated-seed experiments are needed before drawing a general conclusion. For SCCNet and ShallowConvNet, native square activation outperformed all four alternatives. Squaring produces nonnegative power-like features before averaging and logarithmic compression. Replacing it changes this representation; negative pooled outputs from ELU, GELU, or LeakyReLU are clipped by the safe-log operation, potentially discarding information and suppressing gradients. This interaction may contribute to their lower accuracy.

B. ELU and Other Activation Functions

Activation functions introduce nonlinearity into neural networks, allowing them to learn complex relationships between inputs and outputs. Without nonlinear operations, stacking linear layers would still produce a linear mapping, limiting the benefit of network depth. In EEG classification, nonlinear transformations help the model construct discriminative representations from spatially and temporally filtered signals. Their derivatives also affect how gradients propagate during training, influencing optimization behavior.

The exponential linear unit (ELU) was designed to combine a linear positive region with a smooth, negative-valued region [6]:

$$f(x) = \begin{cases} x, & x > 0, \\ \alpha(e^x - 1), & x \leq 0. \end{cases} \tag{1}$$

Our baseline uses $\alpha = 1$. For positive inputs, ELU preserves the input and provides a gradient of one, helping gradients propagate without attenuation from the activation itself. For negative inputs, it produces bounded negative outputs rather than setting them all to zero. This allows activation means to move closer to zero and retains a nonzero gradient for finite negative inputs. However, strongly negative inputs approach the saturation value $-\alpha$, where gradients become small.

These design choices distinguish ELU from the other tested functions. ReLU sets negative inputs to zero, which can leave a unit inactive with zero gradient. LeakyReLU retains a small negative slope, set to 0.01 in our implementation. GELU instead uses $f(x) = x\Phi(x)$ to smoothly weight inputs according to their values [7]. Figure 5 illustrates these differences. Thus,

choosing an activation function changes both the features represented by the network and the gradients used to learn them. No single function is guaranteed to perform best for every model.

C. EEGNet Model Implementation

The PyTorch implementation separates temporal, spatial, and separable convolutional blocks. Inputs have shape $(B, 1, C, T)$, where $C = 2$ electrodes and $T = 750$ samples. Listing 1 contains model excerpts; supporting initialization, constraint, and flattening utilities are defined in the repository.

Listing 1. EEGNet implementation excerpts.

```
# Excerpts from models.py; supporting utilities are defined
    in the repository.
def activation(name):
    return {'elu': nn.ELU, 'relu': nn.ReLU,
            'leakyrelu': lambda: nn.LeakyReLU(0.01), 'gelu':
     nn.GELU,
            'square': Square}[name]()

class SamePad(nn.Module):
    def __init__(self, width):
        super().__init__()
        self.width = width

    def forward(self, x):
        return F.pad(x, ((self.width-1)//2, self.width//2,
    0, 0))

class EEGFeatures(nn.Module):
    def __init__(self, channels, f1=8, d=2, kernel=64,
    dropout=0.5,
                 act='elu', pool1=4, pool2=8):
        super().__init__()
        f2 = f1*d
        self.spatial = nn.Conv2d(f1, f2, (channels, 1),
    groups=f1, bias=False)
        self.temporal = nn.Sequential(SamePad(kernel),
            nn.Conv2d(1, f1, (1, kernel), bias=False),
            nn.BatchNorm2d(f1, eps=1e-3, momentum=0.01))
        self.spatial_block = nn.Sequential(self.spatial,
            nn.BatchNorm2d(f2, eps=1e-3, momentum=0.01),
    activation(act),
            nn.AvgPool2d((1, pool1)), nn.Dropout(dropout))
        self.separable = nn.Sequential(SamePad(16),
            nn.Conv2d(f2, f2, (1, 16), groups=f2, bias=False
    ),
            nn.Conv2d(f2, f2, 1, bias=False),
            nn.BatchNorm2d(f2, eps=1e-3, momentum=0.01),
    activation(act),
            nn.AvgPool2d((1, pool2)), nn.Dropout(dropout))

    def forward(self, x):
        return self.separable(self.spatial_block(self.
    temporal(x)))

class EEGNet(BaseModel):
    def __init__(self, channels=2, samples=750, classes=2,
    dropout=None, act='native', **_):
        super().__init__()
        act = 'elu' if act == 'native' else act
        self.features = EEGFeatures(channels, dropout=0.5 if
     dropout is None else dropout, act=act)
        self.classifier = nn.Linear(16*(samples//4//8),
    classes)
        self.constraints = [(self.features.spatial, (2,),
    1.), (self.classifier, (1,), 0.25)]
        initialize(self)

    def forward(self, x):
        return self.classifier(flatten_channels_last(self.
    features(x)))
```

The activation factory creates a module for each requested nonlinearity. The `SamePad` module pads only time. For temporal kernel width K and unit stride, it adds $\lfloor (K-1)/2 \rfloor$ samples on the left and $\lceil (K-1)/2 \rceil$ on the right. For $K = 64$, these are 31 and 32 samples, preserving temporal length without padding the electrode axis. Temporal convolution produces $(B, 8, 2, 750)$. Spatial depthwise convolution spans both electrodes and produces $(B, 16, 1, 750)$. Pooling by four reduces this to $(B, 16, 1, 187)$. Separable convolution preserves temporal length before pooling by eight, yielding $(B, 16, 1, 23)$. Flattening produces 368 features for the two-class linear classifier. The output logits are passed directly to cross-entropy, without an additional softmax during training.

After each optimizer update, max-norm constraints limit the L2 norm of each spatial filter to 1.0 and each classifier weight vector to 0.25. Weight vectors exceeding these limits are rescaled while preserving their direction. For example, a spatial filter with weights $\mathbf{w} = [0.9, 1.2]$ has an L2 norm of $\sqrt{0.9^2 + 1.2^2} = 1.5$, exceeding the limit of 1.0. Its weights are therefore rescaled as

$$\mathbf{w}_{\text{new}} = \frac{1.0}{1.5}\mathbf{w} = [0.6, 0.8], \tag{2}$$

giving a new L2 norm of 1.0. Weight vectors already within the limit are unchanged. Unlike L2 regularization, this operation directly constrains weight magnitude rather than adding a penalty term to the loss.

D. Depthwise and Separable Convolution

1) Standard and Depthwise Convolution: As illustrated in Figure 6(a)–(b), standard convolution combines all input feature maps for each output, whereas depthwise convolution processes each map independently.

In our EEGNet, temporal convolution produces eight feature maps, each retaining two EEG electrodes. The depthwise spatial layer uses a $(2, 1)$ kernel with `groups=8` and depth multiplier $D = 2$. Each map receives two spatial filters, each learning two electrode weights, resulting in sixteen outputs and $8 \times 2 \times 2 = 32$ weights. A standard convolution producing the same number of outputs would require $16 \times 8 \times 2 = 256$ weights. Depthwise filtering therefore reduces parameters by learning electrode combinations separately within each temporal map [2]. This allows each temporal map to learn its own spatial weighting without mixing information from other maps at this stage.

2) Separable Convolution: EEGNet's separable block consists of two consecutive operations: depthwise temporal convolution with a $(1, 16)$ kernel and `groups=16`, followed by pointwise convolution with a $(1, 1)$ kernel. The first filters each feature map independently. The second combines information across maps, as illustrated in Figure 6(c). Pointwise convolution is therefore only one component of the separable block.

For sixteen input and output maps, depthwise temporal filtering requires $16 \times 16 = 256$ weights, and pointwise mixing requires another $16 \times 16 = 256$. The total is 512 weights, compared with $16 \times 16 \times 16 = 4096$ for a standard convolution with the same temporal kernel. This factorization reduces convolutional weights by 87.5%, contributing to EEGNet's compactness. Its purpose is to separate temporal feature

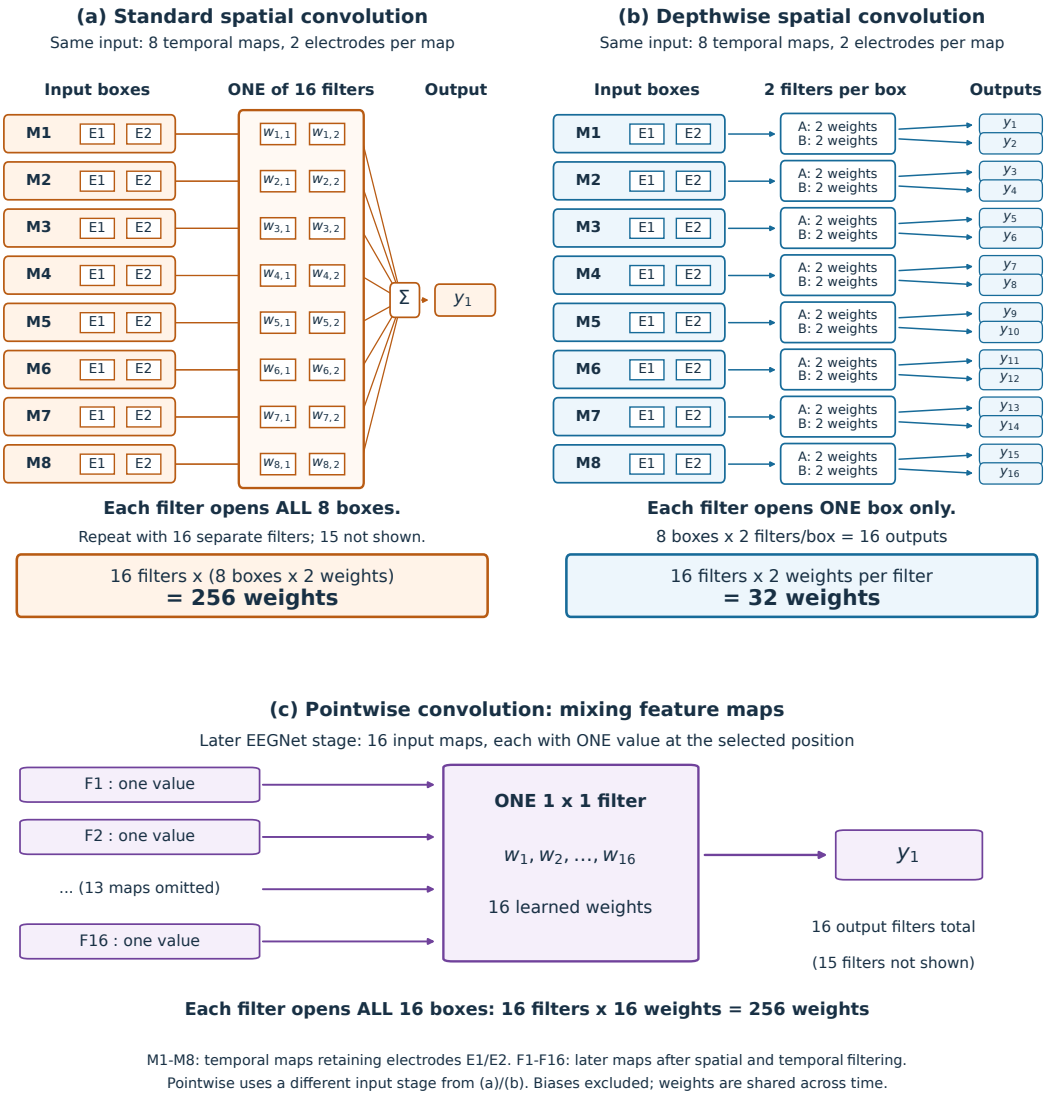


Fig. 6. Connectivity of three convolutional operations. (a) A standard spatial filter combines all eight temporal maps and both electrodes; sixteen outputs require $16 \times 8 \times 2 = 256$ weights. (b) Depthwise spatial convolution processes the maps independently, with two filters per map and two electrode weights per filter: $8 \times 2 \times 2 = 32$ weights. (c) At a later EEGNet stage, pointwise convolution combines sixteen feature-map values at one position using a $(1, 1)$ kernel: $16 \times 16 = 256$ weights for sixteen outputs. Unlike (a)/(b), its input has already undergone spatial collapse and depthwise temporal filtering. Only one output filter is shown in (a)/(c). Biases are excluded, and weights are shared across time.

extraction from cross-map integration, retaining both functions with fewer parameters.

E. Limitations and Future Experiments

The study uses one seed and selects checkpoints and settings by testing accuracy. Peak scores are therefore affected by selection bias and are not unbiased final generalization estimates. Future work should reserve validation data for tuning and report repeated-seed results on an untouched test set. The sequential search also does not exhaust interactions between all hyperparameters.

Another limitation concerns subject-level evaluation. The current experiments pool the training data from S4b and X11b and evaluate on their respective testing data. This measures performance on subjects represented during training, rather than generalization to unseen subjects. Future experiments could compare individual models, trained and tested separately for each subject, with cross-subject models evaluated on a subject excluded from training. This comparison would help assess the benefits of subject-specific learning and the ability to transfer learned EEG representations across individuals. However, with only two subjects, conclusions about cross-subject generalization would remain limited.

Visual cues and feedback are present in the experimental paradigm, but cue-onset alignment is not specified in the NPZ files. A fluctuation in one trial cannot establish a cue-related response or show that the classifier learned cue information. Event markers and the original epoching procedure would

be needed before cue-aligned averaging or controlled pre-cue versus imagery-only comparisons.

IV. GitHub Repository

The source code, installation instructions, and commands for reproducing the experiments are available at: https://github.com/yihmnz/AIMI2026Fall-LAB2.

References

[1] BCI Competition III, "Data set IIIb: Motor imagery with non-stationarity problem," 2005. [Online]. Available: https://www.bbci.de/competition/iii/

[2] V. J. Lawhern, A. J. Solon, N. R. Waytowich, S. M. Gordon, C. P. Hung, and B. J. Lance, "EEGNet: A compact convolutional neural network for EEG-based brain–computer interfaces," *Journal of Neural Engineering*, vol. 15, no. 5, p. 056013, 2018.

[3] R. T. Schirrmeister, J. T. Springenberg, L. D. J. Fiederer, M. Glasstetter, K. Eggensperger, M. Tangermann, F. Hutter, W. Burgard, and T. Ball, "Deep learning with convolutional neural networks for EEG decoding and visualization," *Human Brain Mapping*, vol. 38, no. 11, pp. 5391–5420, 2017.

[4] C.-S. Wei, T. Koike-Akino, and Y. Wang, "Spatial component-wise convolutional network (SCCNet) for motor-imagery EEG classification," in *2019 9th International IEEE/EMBS Conference on Neural Engineering (NER)*, 2019, pp. 328–331. [Online]. Available: https://www.merl.com/publications/docs/TR2018-210.pdf

[5] G. A. Altuwaijri, G. Muhammad, H. Altaheri, and M. Alsulaiman, "A multi-branch convolutional neural network with squeeze-and-excitation attention blocks for EEG-based motor imagery signals classification," *Diagnostics*, vol. 12, no. 4, p. 995, 2022.

[6] D.-A. Clevert, T. Unterthiner, and S. Hochreiter, "Fast and accurate deep network learning by exponential linear units (ELUs)," in *International Conference on Learning Representations*, 2016. [Online]. Available: https://arxiv.org/abs/1511.07289

[7] D. Hendrycks and K. Gimpel, "Gaussian error linear units (GELUs)," *arXiv preprint arXiv:1606.08415*, 2016. [Online]. Available: https://arxiv.org/abs/1606.08415